

Actividad 2. Limpieza de Datos con Meta AI.

Prompt 1. ¿Cuáles son las principales estrategias para tratar valores nulos en un dataset y cómo decides cuál aplicar? Específicamente, ¿cuándo es correcto imputar con una categoría centinela como "Unknown", cuándo eliminar el registro y cuándo conservarlo con un valor derivado?

Aquí va el framework que uso yo en producción:

1. Diagnostica primero el tipo de "missing"

Esto define todo lo demás. Hay 3 tipos:

Tipo	Qué significa	Ejemplo	Estrategia base
MCAR - Missing Completely at Random	El nulo no tiene nada que ver con los datos	Sensor falló 2% de veces aleatoriamente	Casi cualquier método vale
MAR - Missing at Random	El nulo se explica por otras columnas	"Ingreso" falta más en gente joven	Imputar con modelo usando otras columnas
MNAR - Missing Not at Random	El nulo es información en sí mismo	No contestó "Salario" porque gana mucho	Conservar: el hueco tiene señal

Si no sabes cuál es, asume MNAR por defecto. Es lo más seguro.

2. Las 3 estrategias principales y cuándo usar cada una

A. Eliminar el registro - `dropna()`

Cuándo es correcto:

1. **Faltan muy pocos:** <1-5% de filas y son MCAR. No sesgas nada y te quitas ruido.
2. **Nulos en la variable target:** Si no sabes qué quieres predecir, esa fila no sirve.
3. **Nulos en columnas críticas:** Si "user_id" o "fecha_transacción" es nulo, el registro está corrupto.

Cuándo NO:

Si eliminas 30% de datos, destruyes tu dataset. Si los nulos son MNAR, eliminas justo los casos interesantes y metes sesgo.

B. Imputar con categoría centinela - "Unknown", -999, "Missing"

Cuándo es correcto:

1. **MNAR evidente:** El hecho de que falte es predictivo. Ej: "Años de experiencia" nulo en CVs = candidato sin experiencia
2. **Variables categóricas en modelos de árboles:** Random Forest, XGBoost, LightGBM manejan "Unknown" como una cate
3. **Quieres preservar el tamaño del dataset:** Para dashboards o conteos donde cada fila cuenta como un usuario/evento.

Reglas clave:

- Usa un valor imposible en el dominio real: `999` para edad, `1900-01-01` para fechas, `"Unknown"` para texto.
- Documenta el centinela. Si no, otro analista pensará que -999 es una edad real.
- **Cuidado con modelos lineales:** Si pones -999 en regresión lineal, distorsiona la pendiente. Ahí mejor usar media + crear

C. Conservar con valor derivado - imputación

Cuándo es correcto:

1. **Media/Mediana/Moda:** Variable numérica, MCAR o MAR, <10% nulos, sin outliers fuertes. Usa mediana si hay outliers. F

Code

Ej: `Edad faltante -> poner mediana de edad por país + género`

1. **Moda/Frecuente:** Categóricas con una clase dominante. Ej: "País" nulo → "México" si 90% de usuarios son de México.
2. **Imputación por grupo:** Lo más robusto. Usa otras columnas.

Code

```
Ej: Salario nulo → mediana de salario para mismo "Puesto" + "Años_exp"
```

1. **Modelos predictivos - KNN, MICE, Regresión:** Para MAR con muchos nulos 10-30%. Usas el resto de columnas para predecir el valor faltante. Es lo más preciso pero lento.
2. **Interpolación:** Solo para series de tiempo. El valor de ayer y mañana te dicen el de hoy.

3. Regla de decisión rápida

Code

```
% Nulos por columna → ¿Qué hacer? - <1%: Casi lo que sea. Mediana/moda es ok. - 1-5%: Imputación simple o centinela si es MNAR. - 5-30%: Imputación por grupo o modelo. Considera crear columna `col_es_nulo`.- >30%: ¿Esta columna sirve? Si sí, solo centinela. Si no, tira la columna.- >60%: Tira la columna, salvo que el nulo sea la señal principal.
```

1 línea oculta

4. Truco pro: "Missing Indicator"

Cuando imputas, siempre pregúntate: ¿perdí información? La mejor práctica es hacer ambas cosas:

1. Imputa la columna: `edad = edad.fillna(media_edad)`
2. Crea flag: `edad_es_nula = 1 si edad era nula, 0 si no`

Así el modelo aprende que "tener edad imputada" es distinto a "tener edad real". XGBoost hace esto automático, pero en regresión lineal lo tienes que hacer tú.

Resumen: ¿Cuál elijo?

Situación	Estrategia	Por qué
Encuesta: no respondió "Ingresos"	Centinela "No_declarado"	Es MNAR: gente rica/pobre no dice
Sensor IoT falló 0.5% lecturas	Eliminar filas o interpolar	Es MCAR y son pocas
"Fecha_segunda_compra" nula	Centinela "Sin_segunda_compra"	El nulo = cliente no volvió. Oro para churn
"Temperatura" nula en serie temporal	Interpolación	El valor anterior y siguiente son buena pista
40% nulos en columna "LinkedIn_URL"	Tirar columna o usar solo como flag binario	Demasiado hueco, poco valor real

Regla de oro: Si el nulo significa algo para tu problema de negocio, conviértelo en feature. Si es solo ruido/error, imputa o elimina.

Conclusión: El correcto manejo de los datos nulos/corruptos/faltantes es de suma importancia para el análisis de datos pues en algunos casos, como el uso de algoritmos de IA, son necesarios para el entrenamiento y la validación. Una regla importante que hay que denotar es que cuando el 30% de los datos de una base de datos son "tirados" esta pierde su utilidad y puede perder su calidad en la investigación realizada.

Prompt 2. ¿Cuál es el papel de las Expresiones Regulares (REGEX) dentro de un pipeline de limpieza de datos con Python y Pandas? ¿En qué situaciones son indispensables frente a métodos más simples como `.str.replace()` o `.str.strip()`? Proporciona 2 ejemplos de patrones REGEX comunes en la limpieza de texto real.

¿Cuál es su papel en un pipeline con Python + Pandas?

En Pandas usas regex principalmente a través de `.str` con `extract`, `replace`, `contains`, `split`, `findall`. Su papel es:

1. **Validación y filtrado de patrones:** Identificar filas que cumplen/no cumplen un formato específico.

Python

```
df[df['email'].str.contains(r'^[\w\.-]+\@[\w\.-]+\.\w+$', na=False)]
```

1. **Extracción estructurada de datos no estructurados:** Sacar trozos específicos de strings caóticos y convertirlos en columnas nuevas.

Python

```
df['codigo_postal'] = df['direccion'].str.extract(r'(\d{5})')
```

1. **Transformación compleja:** Reemplazos que dependen del contexto, no de texto literal.

Python

```
df['telefono'] = df['telefono'].str.replace(r'^\d', '', regex=True) # deja solo números
```

1. **Normalización masiva:** Estandarizar miles de variantes a un formato único en una sola pasada.

¿Cuándo son indispensables vs `.str.replace()` o `.str.strip()` ?

Usa métodos simples cuando el problema es literal. Usa regex cuando el problema es *estructural*.

Situación	<code>.str.strip()</code> / <code>.replace()</code>	REGEX indispensable
Quitar espacios	<code>df['col'].str.strip()</code> quita espacios al inicio/final	Quitar espacios extra <i>entre</i> palabras: <code>df['col'].str.replace(r'\s+', ' ', regex=True)</code>
Reemplazo fijo	<code>df['col'].str.replace('\$', '')</code> quita signos de dólar	Quitar <i>cualquier</i> símbolo que no sea letra/número: <code>df['col'].str.replace(r'^a-zA-Z0-9\s', '', regex=True)</code>
Validar formato	Imposible	Verificar si un RFC, CURP, email tiene estructura válida: <code>df['rfc'].str.match(r'^[A-Z]{4}\d{6}[A-Z0-9]{3}\$')</code>
Extraer datos	Imposible si no conoces el texto exacto	Sacar precios de "Producto \$1,299.00 MXN": <code>df['texto'].str.extract(r'\\$?([\d,]+\.\d*)')</code>
Lógica condicional	Imposible	Reemplazar solo si la palabra está completa: <code>r'\bSr\.\b'</code> no toca "Señora"

Regla rápida: Si puedes describir lo que buscas con "la palabra exacta X", usa `.replace()`. Si necesitas "cualquier cosa que se vea como X", usa regex.

2 patrones REGEX que vas a usar todo el tiempo en datos reales

1. Limpiar números de teléfono / dejar solo dígitos

Datos reales: (55) 1234-5678 , 55-1234-5678 , +52 55 1234 5678 , 5512345678 ext 123

Python

```
df['telefono_limpio'] = df['telefono'].str.replace(r'\D', '', regex=True)
```

- `\D` = cualquier carácter que *no* sea dígito 0-9
- Resultado: `5512345678` para todos los casos de arriba
- Con `.replace()` tendrías que encadenar 10 reemplazos: `'-', '(', ')', ' ', '+', 'ext'...` y siempre se te escapa uno.

2. Extraer dominios de emails y normalizar

Datos reales: Juan.Perez@Gmail.COM , ana--lopez@yahoo.com.mx , soporte@empresa.co

Python

```
# Paso 1: limpiar y extraerdf['email'] = df['email'].str.strip().str.lower()df['dominio'] = df['email'].str.extract(r'@([\w\.-]+)')
# Paso 2: detectar emails malformadosemails_invalidos = df[~df['email'].str.match(r'^[\w\.-]+\@[\w\.-]+\.\w+$', na=False)]
```

1 línea oculta

- `[\w\.-]+` = uno o más caracteres de palabra, punto o guión
- `@([\w\.-]+)` = captura todo después del @ como grupo
- Resultado: `gmail.com` , `yahoo.com.mx` , `empresa.co`

Bonus: Quitar múltiples espacios + espacios al inicio/final en una línea:

Python

```
df['nombre'] = df['nombre'].str.replace(r'^\s+|\s+(?=\s)', '', regex=True)
```

" Juan Pérez " → "Juan Pérez"

¿Cuándo NO usar regex?

1. Reemplazo literal simple: `df['col'].str.replace('N/A', '')` es más rápido sin `regex=True`
2. Dividir por delimitador fijo: `df['col'].str.split(',')` vs `re.split(r',')`
3. Si tu equipo no lo entiende: regex es "write-only" si te pasas de complejo

Regex tiene costo: es más lento y menos legible. Pero cuando limpias datos del mundo real - PDFs scrapeados, formularios libres, logs - es la diferencia entre pasar 3 horas en loops vs 3 líneas de código.

Conclusión: REGEX permite: quitar espacios, reemplazar objetos, validar formatos, extraer datos y aplicar lógica condicional sobre el dataframe. Esto en búsqueda de procesar los datos en crudo y ajustarlos a un dataframe limpio y fácilmente manejable.

Prompt 3. ¿Cómo impacta la correcta asignación de tipos de datos (como int, float, category u object) en el rendimiento de un DataFrame de Pandas? ¿Por qué es importante convertir columnas categóricas al tipo category en lugar de dejarlas como object, especialmente cuando el dataset tiene millones de registros?

Cuando cargas un DataFrame, Pandas adivina los tipos. Y adivina conservador: todo lo que parezca texto lo deja como `object`. Funciona, pero es carísimo en memoria y CPU. Elegir bien el tipo = menos RAM + operaciones 10x-100x más rápidas.

Pandas está construido sobre Numpy. Numpy guarda arrays contiguos de bytes en C. Eso es lo que hace que sea rápido. C:

Cuando dejas todo como `object`

`object` significa "arreglo de punteros de PyObject*". Cada celda del DataFrame no contiene el dato, contiene una dirección

1. Cómo impacta cada tipo en el rendimiento

Tipo	Qué guarda internamente	Memoria por valor	Operaciones vectorizadas	Ejemplo
<code>int64</code>	8 bytes fijos	8 bytes	Ultra rápidas en C	IDs, conteos
<code>int32</code> , <code>int16</code> , <code>int8</code>	4, 2, 1 byte	4/2/1 bytes	Muy rápidas	Edad, mes 1-12
<code>float64</code>	8 bytes fijos	8 bytes	Rápidas en C	Precios, lat/lon
<code>float32</code>	4 bytes	Rápidas, mitad RAM	ML, embeddings	
<code>object</code>	Puntero a objeto Python	8 bytes + objeto heap	Lentas, pasa a Python	Texto libre
<code>category</code>	Códigos int8/int16 + dict	1-2 bytes + dict	Muy rápidas para groupby	País, género
<code>bool</code>	1 byte	1 byte	Rapidísimas	Flags true/false
<code>datetime64[ns]</code>	int64 de nanosegundos	8 bytes	Optimizadas	Fechas

Regla clave: `object` rompe la vectorización. Cada vez que haces `.sum()`, `.groupby()` o `.str` en una columna `object`, Pandas tiene que llamar al intérprete de Python por cada fila. Con `int/float/category` usa código C/Numpy compilado.

2. Por qué `category` vs `object` es brutal con millones de filas

Supón una columna `país` con 5M de filas pero solo 200 países únicos:

Como `object`

Code

```
Memoria: 5,000,000 * (8 bytes puntero + ~50 bytes string promedio) = 290 MB
Groupby: Tiene que hacer hash de 5M strings de Python.
sort_values(): Compara strings en Python, lentísimo
```

Cada string es un objeto Python completo en el heap. El DataFrame solo guarda punteros.

Como **category**

Code

```
Memoria: 5,000,000 * 1 byte código + dict de 200 strings ≈ 5 MB + dict ≈ 5.1 MB
Groupby: Opera sobre enteros de 1 byte, no strings.sort_values(): Compara enteros, 50x más rápido
```

Pandas guarda la columna como códigos `int8` 0-199 y un diccionario `{0: 'México', 1: 'USA', ...}`.

Ganancia real: 290 MB → 5 MB = 98% menos RAM. Un `.groupby('pais').mean()` pasa de 2.3s a 0.04s.

3. Cuándo convertir a **category**

Hazlo cuando se cumplen 2 condiciones:

1. **Pocos valores únicos:** Cardinalidad baja. Regla: si `n_únicos / n_filas < 5%`, ganas.

- Bien: `género`, `estado`, `tipo_producto`, `código_postal`
- Mal: `user_id`, `email`, `descripción`

2. **Usas la columna para agrupar, filtrar o hacer joins:** `groupby`, `merge`, `value_counts`, `isin`.

Ojo: Si solo vas a imprimir la columna o hacer `.str.contains()`, `category` puede ser más lento porque tiene que mapear código → string.

4. Ejemplo rápido con números

Python

```
import pandas as pd
import numpy as np
n = 10_000_000
df = pd.DataFrame({'pais': np.random.choice(['MX', 'US', 'BR', 'CA', 'AR'], n), 'valor': np.random.randn(n)})
# Como object: ~80 MB df['pais'].memory_usage(deep=True) / 1e6
# Convertir a category df['pais'] = df['pais'].astype('category') # Como category: ~10 MB df['pais'].memory_usage(deep=True) / 1e6
%timeit df.groupby('pais')['valor'].mean() # object: 1.2 s # category: 0.03 s -> 40x más rápido
```

15 líneas ocultas

5. Otras conversiones que importan

1. `int64` → `int32/int16`: Si sabes que una edad nunca pasa de 120, usa `int8`. Ahorras 87% RAM.
2. `float64` → `float32`: Para ML es estándar. Mitad de RAM, casi misma precisión.
3. `object` → `string[pyarrow]`: Desde Pandas 2.0, el dtype `string` usa menos RAM que `object` y soporta nulos mejor.

Resumen: Con datasets grandes, los tipos mal asignados te comen la RAM y matan la CPU. `object` es el enemigo para columnas repetitivas. `category` convierte un problema de `O(n)` strings en `O(k)` donde k = valores únicos.

Si tu DataFrame no cabe en RAM, lo primero que debes revisar son los `dtypes` antes de pensar en Dask o Spark.

Cuando Pandas crea una columna, lo que realmente está decidiendo es cómo va a acomodar esos valores en la memoria RAM. Si eliges `int64` o `float64`, Pandas le pide a NumPy un bloque continuo de memoria donde cada valor ocupa exactamente ocho bytes, uno tras otro, sin huecos. La CPU puede tomar ese bloque y recorrerlo con instrucciones vectorizadas en C, sumando, comparando o filtrando sin tener que volver a Python ni una sola vez. Es como leer un libro donde cada palabra tiene el mismo ancho, puedes pasar la vista a toda velocidad.

Cuando dejas una columna como `object`, que es lo que Pandas hace por defecto con cualquier texto, no guarda el texto ahí. Guarda un arreglo de punteros, y cada puntero apunta a un objeto Python suelto en otra parte de la memoria. Ese objeto tiene su propio encabezado, su contador de referencias, su codificación unicode. Entonces cada vez que quieres contar cuántas veces aparece "México", Pandas tiene que seguir el puntero, entrar al intérprete de Python, calcular el hash de ese string, compararlo carácter por carácter, y repetir eso millones de veces. No hay forma de vectorizar porque el intérprete no sabe dónde termina un objeto y empieza el siguiente.

Ahí es donde entra `category`. No es magia, es una indirección controlada. Pandas mira tu columna, extrae la lista de valores únicos una sola vez y la guarda en un diccionario pequeño. Luego reemplaza cada aparición original por el índice de ese diccionario, que es un entero diminuto. Si tienes diez millones de filas pero solo doscientos países distintos, en lugar de

guardar diez millones de strings guarda diez millones de números entre cero y ciento noventa y nueve. Eso cabe en un `int8`, un solo byte por fila. El diccionario con los doscientos nombres se guarda aparte, una sola vez.

El impacto con millones de registros se siente en dos lugares que duelen mucho. El primero es la memoria. Con `object`, diez millones de strings cortos pueden llevarte fácilmente a seiscientos megabytes solo en esa columna, porque cada string cuesta cincuenta o sesenta bytes más el puntero. Con `category` esa misma columna baja a diez megabytes. Cuando tu DataFrame cabe en memoria, todo es rápido. Cuando no cabe, el sistema operativo empieza a paginar a disco y tu análisis que tardaba segundos pasa a tardar minutos. El segundo lugar es la caché del procesador. La caché L3, que es la más grande, ronda los diez o veinte megabytes. Un arreglo de diez millones de bytes entra entero y la CPU lo recorre sin salir a RAM. Un arreglo de punteros de ochenta megabytes no entra, y cada acceso provoca un fallo de caché que cuesta cien ciclos.

Por eso las operaciones que más usas se aceleran tanto. Un groupby sobre una columna `object` tiene que hashear strings en Python para saber a qué grupo pertenece cada fila. Un groupby sobre `category` ya tiene el grupo codificado como un entero, así que puede usar una rutina en C que simplemente incrementa un contador en la posición de ese entero. Lo mismo pasa con un filtro, con un sort, con un merge. Comparar dos enteros es una instrucción de CPU. Comparar dos strings es una función de Python que revisa longitudes, normaliza unicode y recorre bytes.

El tipo numérico también importa por la misma razón. Mucha gente deja todo en `int64` porque es el default. Si estás guardando edades, meses o cantidades pequeñas, un `int64` sigue usando ocho bytes por valor aunque nunca pases de cien. Cambiar a `int8` o `int16` no cambia la velocidad de cálculo, porque sigue siendo un bloque continuo, pero reduce la memoria a la mitad o a un octavo. Con floats pasa algo parecido. `float64` te da quince dígitos decimales, `float32` te da siete, que suele ser más que suficiente para precios o coordenadas, y te ahorra la mitad de RAM. Esa memoria liberada es la que te permite mantener el DataFrame en caché.

Convertir a `category` no siempre ayuda. Si tu columna tiene casi tantos valores distintos como filas, por ejemplo un identificador de usuario o un email, el diccionario de categorías se vuelve tan grande como la columna original y además pagas el costo de traducir cada vez que quieras hacer una operación de texto como buscar una subcadena. En esos casos `object`, o mejor aún el nuevo dtype `string` con backend pyarrow, es más honesto. La ganancia real aparece cuando hay repetición, cuando usas esa columna para agrupar, para filtrar por igualdad, para hacer joins. Ahí la repetición es la clave, porque estás reemplazando trabajo repetido sobre strings por trabajo repetido sobre enteros.

En la práctica, con un dataset grande, la asignación correcta de tipos es lo que separa un notebook que responde al instante de uno que se queda colgado. No es un detalle de estilo, es la forma en que le dices a Pandas si quieres que trabaje con bloques de memoria simples en C o si quieres obligarlo a entrar al intérprete de Python fila por fila. Y con millones de filas, esa decisión se multiplica por millones.

Conclusión: El uso de los recursos computacionales para guardar “temporalmente” los data frames mientras se manipulan permite mejorar el procesamiento computacional de las consultas, y posteriormente las funciones aplicadas sobre estos datos. Es de considerarse esta manipulación y transformación de datos pues con grandes volúmenes de datos es considerable el tiempo de ejecución del análisis por proceso.